

exp-mlkem-f203-stage1-encode-vec

F203 Stage1 纯向量实现方案（API 全覆盖评估）

ascendc 工程（预研）

2026-05-19

摘要

本文档描述 `examples/incubating/exp-mlkem-f203-stage1-encode-vec/` 用例：将 F203 交付件 `sepolyvec8_ntt_f203` 的 Stage1 (`se [8,256] int32 → mat_a [16,256] int8`, 布局 $[HI(8), LO(8)]$) 在 Atlas A2 / Ascend910B 上以 **纯 AIV 向量** 实现。API 依据 CANN 社区版 9.0.0 Ascend C 算子开发接口参考 01 (PDF 第 2 章 SIMD API) 及国腾社区在线页; `dav_c220` (910B) 头文件 `kernel_operator_vec*_impl.h` 用于交叉核对 A2 上 `Cast` 等组合是否真实可用。CPU 对拍: `bash run.sh -r cpu -v Ascend910B4; 1 / 2 / 8 个 Vector AI Core` 三档 (见 §2.1)。

目录

1 数学语义	2
1.1 输入输出	2
1.2 逐系数公式 (对齐 <code>MlkemEncodeToLimb6</code> / 交付 <code>Floor-Sub</code> 在 $v \geq 0$ 时的等价)	2
1.3 Golden	2
2 向量实现规划	2
2.1 AI Core 规模与验证档位	2
2.2 并行与分块	3
2.3 计算步骤与 API 映射	3
3 全量 API 清单 (查阅 CANN 9.0.0)	3
3.1 曾评估但未采用的 API	4
4 Cast 窄化链与备选	5
4.1 目标	5
4.2 A2 文档表 6 允许的组合 (摘录)	5
4.3 实现链 (已验证)	5
4.4 备选方案 (若 <code>half</code> 链不可用)	5
5 逐步代入与覆盖率评估	5
6 源码与验证	6

1 数学语义

1.1 输入输出

- 输入 `se_polyvec_gm`: 形状 $[8, 256]$, `int32`。Host/Python 保证每条系数 $v \in [0, q)$, $q = 3329$ 。
- 输出 `mat_a_gm`: 形状 $[16, 256]$, `int8`。行 0..7 为 *HI*, 行 8..15 为 *LO*。
- 本阶段不做 $\bmod q$ (与交付 ONNX 一致; Python `validate_se_polyvec_in_zq` 保证前提)。

1.2 逐系数公式 (对齐 `MlkemEncodeToLimb6` / 交付 `Floor-Sub` 在 $v \geq 0$ 时的等价)

对每条 poly、每个系数 v :

$$hi = \lfloor v/64 \rfloor = v \gg 6 \quad (1)$$

$$lo = v - hi \cdot 64 \quad (\text{在 } v \in [0, q) \text{ 时等价于 } v \bmod 64) \quad (2)$$

不采用 $lo = v \wedge 63$ 的独立 And 路径作为最终实现: CPU 仿真上曾出现 `lo` 错误并导致 `Cast` 饱和到 127 (见 §3.1)。

1.3 Golden

`mlkem_ref.encode_mat_a(se):hi_i8 = (se>>6).astype(int8),lo_i8 = (se&63).astype(int8)`, 沿 `axis=0` 拼接。

2 向量实现规划

2.1 AI Core 规模与验证档位

项	本用例约定
核类型	<code>KERNEL_TYPE_AIV_ONLY</code> : 仅使用 Vector AI Core (AIV) , 不使用 Cube AI Core (AIC) 。
计划 AI Core 数	三档: $aiv=1 \Rightarrow blockDim=1$; $aiv=2 \Rightarrow blockDim=2$; $aiv=8 \Rightarrow blockDim=8$ (<code>launch_profile.h</code>)。
数据划分 ($aiv=8$)	<code>GetBlockIdx()==i</code> : 每核 1 poly; 读 <code>se</code> 第 i 行; 写 <code>mat_a</code> <i>HI/LO</i> 第 $i/8+i$ 行。
数据划分 ($aiv=2$)	<code>blockIdx=0,1</code> : 每核 4 poly; 核内 for $p = 4 \cdot blockIdx..4 \cdot blockIdx + 3$ 串行; 每 poly 新建 op 。
数据划分 ($aiv=1$)	<code>GetBlockNum()==1</code> : 核内 for $p = 0..7$ 串行; 每 poly 新建 op (避免 <code>TPipe/TQue</code> 跨 poly 复用)。
多档验证	默认三档均测 : <code>run.sh</code> 依次 $aiv=1$ 、 $aiv=2$ 、 $aiv=8$; 亦可 <code>--aiv 1 2 8</code> 单档。

融合算子 AIV 约束	含 Cube 的融合内核 (如 <code>sepolyvec8_ntt_k4</code>) 在 910B 上 Vector 数须为 2 的倍数 (典型 MIX 1 AIC : 2 AIV)。本用例纯 AIV, 增测 <code>aiv=2</code> 以验证 最小偶数多核 分核与流水, 便于与融合 launch 对齐。
验证命令	Python 先生成 <code>output/golden.bin</code> ; 每档 launch 后 <code>md5sum + cmp + verify_result.py</code> 。

选型理由:

- **aiv=8**: $K=8$ 按 poly embarrassingly parallel, 满核数据并行。
- **aiv=2**: 每核 4 poly; 覆盖融合路径要求的**偶数 AIV** 最小规模, 验证分核偏移与多 poly 串行。
- **aiv=1**: 单核 golden 基线, 便于 Stage 隔离联调。
- 核入口: `blockNum==1` 串行 8 poly; 否则 `polysPerBlock=8/blockNum, startPoly=blockIdx*polysPerBlock`, 核内逐 poly 新建 op。

2.2 并行与分块

- 核规模见 §2.1; 每 poly 仍为 256 系数、`kTileLength=32` 流水。
- TPipe + TQue 双缓冲: `kTileNum=4, kBufferNum=2` $\Rightarrow$ `kTileLength=32`。
- 流水: `CopyIn` $\rightarrow$ `Compute` $\rightarrow$ `CopyOut`, 与 `AddCustomSample` 同构。

2.3 计算步骤与 API 映射

1. **GM** $\rightarrow$ **UB**: `DataCopy` 读 se 一 tile (`int32`)。
2. 求 *hi*: `ShiftRight(hi, v, 6)`。
3. 求 *lo*: `Muls(tmp, hi, 64); Sub(lo, v, tmp)`。
4. 窄化 `int8`: `CastI32ToI8` 链 (§4)。
5. **UB** $\rightarrow$ **GM**: `DataCopy` 写 HI/LO 行。

3 全量 API 清单 (查阅 CANN 9.0.0)

说明: 分类按 PDF 第 2 章; 在线页为社区 `atlasascendc_api_07_*`; PDF 约页据目录 (2.3.1 自 p.136, 2.3.3 自 p.357, 具体 API 页码因文档排版略有偏差, 以在线页为准)。

API	分类	在线文档 ID	A2	输入/输出 dtype (本用例)	备注
GlobalTensor	2.2 数据结构	07_0007	✓	GM 张量描述	SetGlobalBuffer
LocalTensor	2.2 数据结构	07_0006	✓	UB 张量描述	AllocTensor/Get
GetBlockIdx	2.3 系统变量	07_0185	✓	返回核索引	int64_t
KERNEL_TASK_TYPE_DEFAULT	Utils / 核类型	见编程指南	✓	KERNEL_TYPE_AIV_ONLY 纯向量核	
TPipe	2.3 资源管理	07_0108	✓	管道对象	每 kernel 唯一实例
InitBuffer	2.3 资源管理	07_0110	✓	为 TQue/TBuf 分配 UB	TPipe 成员
TQue	2.3 Pipe/Que	07_0137	✓	队列; VECIN/VEECOUT	EnQue/DeQue
TBuf	2.3 Pipe/Que	07_0137 同章	✓	VECCALC 临时区	Get/GetWithOffset
DataCopy	2.3.1 搬运	07_0101	✓	int32/int8 GM↔UB	连续块; 本用例对齐
ShiftRight	2.3.3 矢量	07_0059	✓	src/dst: int32_t; 标量 [0, 32]	算术右移
Muls	2.3.3 矢量	07_0055	✓	src/dst: int32_t; 标量 int32	$hi \times 64$
Sub	2.3.3 矢量	07_0036	✓	src/dst: int32_t	$v - hi \cdot 64$
Cast	2.3.3 类型转换	07_0073	✓	见 §4	分步窄化

3.1 曾评估但未采用的 API

API	在线 ID	A2	原计划	结论
And	07_0062 附近	✓	$lo = v \wedge 63$	int16_t/uint16_t; 非 int32。且 CPU 路径曾未得到正确 lo
Cast(int32→int8)	07_0073	×	直接窄化	A2 表 6 无此组合; dav_c220 CheckCastDatatype 拒绝

Cast(int16→ 07_0073 × 二级窄化 同上; CPU 仿真 abort
int8)

4 Cast 窄化链与备选

4.1 目标

将 $hi, lo \in [0, 63]$ 的 int32 写入 int8 输出 (值域安全, 无饱和风险)。

4.2 A2 文档表 6 允许的组合 (摘录)

- int32_t → int16_t: CAST_NONE
- int16_t → half: CAST_NONE (或与 SetDeqScale 配合)
- half → int8_t: 多种 RoundMode

4.3 实现链 (已验证)

```
Cast(tmpI16, srcI32, CAST_NONE, count);
Cast(tmpHalf, tmpI16, CAST_NONE, count);
Cast(dstI8, tmpHalf, CAST_NONE, count);
```

4.4 备选方案 (若 half 链不可用)

1. **备选 A:** int32→float→int8 (需核对表 6 是否含 int32→float 与 float→int8; 有舍入语义)。
2. **备选 B:** 标量循环写 int8 (仅 32 元/tile, 可接受但失去向量窄化)。
3. **备选 C:** 查 MulCast 等复合 API (本用例值域小, 非必须)。

5 逐步代人与覆盖率评估

步骤	数学	API	覆盖
S0	核映射	GetBlockIdx, KERNEL_TASK_TYPE_DEFAULT	100%
S1	读 v	DataCopy + TQue	100%
S2	$hi = v \gg 6$	ShiftRight	100%
S3	$lo = v - 64 \cdot hi$	Muls, Sub	100%
S4	写 int8	Cast×3	100% (链式)
S5	写 GM	DataCopy	100%
合计 (主路径)			100% 向量

结论：在采用 $lo = \text{Sub}(v, \text{Muls}(hi, 64))$ 与三段 Cast 前提下，Stage1 主路径 **无需标量回退**。标量仅存在于核 launch 壳 (`main.cpp`) 与 Python Golden。

6 源码与验证

- 内核: `f203_stage1_encode_custom.cpp`
- Launch: `launch_profile.h` (`aiv=1/aiv=8`)
- 验证: `bash run.sh -r cpu -v Ascend910B4` (默认两档; golden md5sum 对拍)
- 索引: `library/documents/CANN-AscendC` 算子开发接口参考-查阅索引.md (2026-05-19 条目)